# Welcome to cocotb

Python can drive your testbench while the simulator owns time and the DUT

# Why cocotb

- SystemVerilog testbenches are powerful
- More, while still talking to real RTL
- This course builds literacy you can reuse with Verilator or Icarus

## Two tracks, one idea

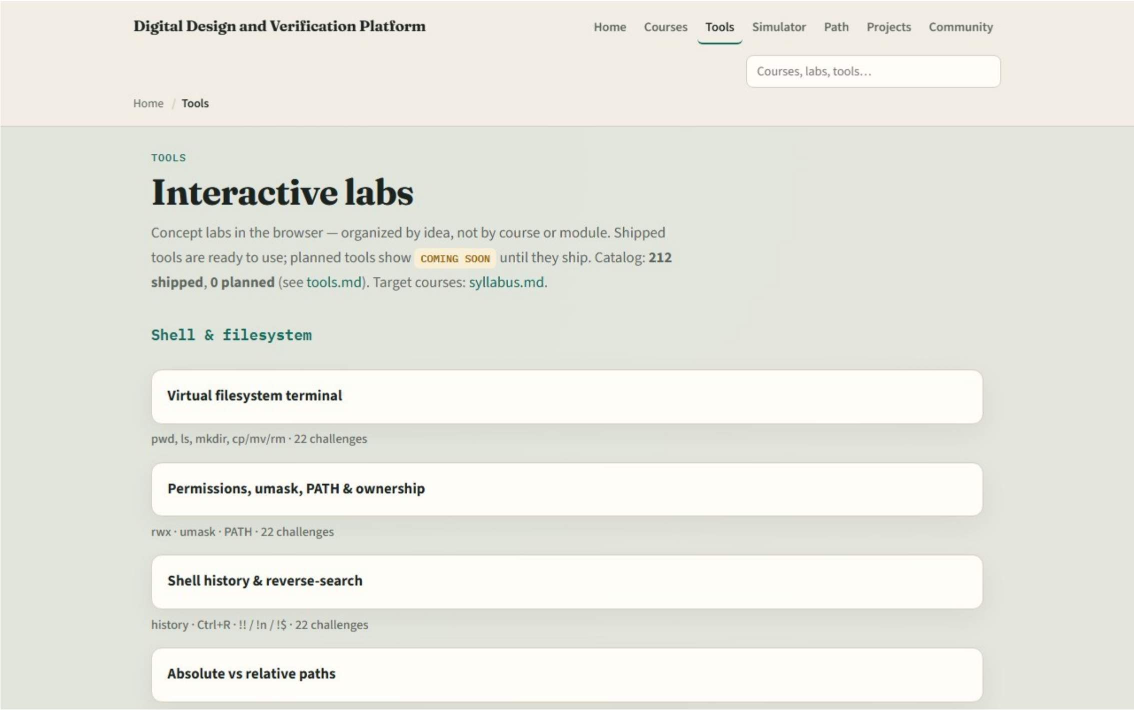
- Every lab module offers two ways to practice
- Makefile runs feel like work you will keep
- Waveform literacy, so you can build intuition without finishing every install first
- You may do either track, or both

# Set up the real cocotb track

- Install Python three, cocotb, and a free simulator, Verilator is a common choice
- Open this course in the monorepo and skim the two-track doc for exact tool names
- You do not need a perfect farm on day one; module ten is the dedicated offline run
- For now, confirm you know where the examples prompts live in each module folder

# Set up the browser lab track

- From the monorepo
- Look for the cocotb sketches, async testbench, triggers, DUT handle, scoreboard
- If you prefer, use the published tools site instead
- Either way, open one sketch once so you know the layout



# How to move through modules

- For each module
- Browser labs teach concepts; the real track keeps fidelity
- When you finish this intro checklist, continue to the first lab: Python async testbench